



Universidad Veracruzana

Proyecto mínimo en Django

Instalación, rutas y autenticación de usuarios

Programación segura

Dr. Juan Manuel Gutiérrez Méndez

Propósito de la sesión

Construir una primera versión mínima de un proyecto Django con autenticación de usuarios y acceso protegido a pantallas.

En esta primera versión no se registrará información del dominio. El objetivo es verificar la estructura base, las rutas, el administrador, el login y la protección de vistas.

Al finalizar, el estudiante deberá poder:

- crear un proyecto Django desde cero;
- crear una sola aplicación llamada `bienes` ;
- configurar rutas principales y rutas de aplicación;
- habilitar el administrador;
- crear usuarios;
- construir una pantalla de inicio de sesión;
- proteger pantallas mediante autenticación.

Ruta de trabajo para implementar django

Versión 1

Proyecto mínimo con autenticación, rutas y pantallas protegidas.

Versión 2

Incorporación de modelo, formulario y registro de bienes.

Versión 3

Capa de seguridad con validadores reutilizables para formularios.

La construcción será incremental para que cada responsabilidad arquitectónica pueda identificarse y probarse por separado.

Proyecto que se construirá

Se desarrollará una base mínima para una aplicación de gestión de bienes.

```
proyecto_seguro/  
  manage.py  
  
  config/  
    settings.py  
    urls.py  
  
  apps/  
    __init__.py  
    bienes/  
      __init__.py  
      apps.py  
      urls.py  
      views.py  
      templates/  
        bienes/  
          lista.html  
          formulario.html  
  
  templates/  
    base.html  
    registration/  
      login.html
```

En esta versión, `bienes` tendrá rutas, vistas y plantillas, pero todavía no tendrá modelo ni formulario de captura.

Paso 1: crear carpeta de trabajo

Desde la terminal, crear una carpeta para el proyecto.

```
mkdir proyecto_seguro  
cd proyecto_seguro
```

Crear un entorno virtual.

```
python -m venv .venv
```

Activar el entorno virtual en Windows PowerShell.

```
.venv\Scripts\Activate.ps1
```

Activar el entorno virtual en Linux o macOS.

```
source .venv/bin/activate
```

El entorno virtual separa las dependencias del proyecto y evita mezclar versiones de paquetes entre prácticas.

Paso 2: instalar Django

Instalar Django en el entorno virtual.

```
python -m pip install --upgrade pip  
python -m pip install Django==6.0.5
```

Verificar la instalación.

```
python -m django --version
```

Guardar dependencias.

```
python -m pip freeze > requirements.txt
```

La versión instalada debe mantenerse consistente entre el código, la práctica, la documentación y el entorno de cada estudiante.

Paso 3: crear el proyecto principal

Crear el proyecto Django usando `config` como carpeta principal de configuración.

```
django-admin startproject config .
```

Estructura generada:

```
proyecto_seguro/  
  manage.py  
  config/  
    __init__.py  
    settings.py  
    urls.py  
    asgi.py  
    wsgi.py
```

Archivo	Responsabilidad
<code>manage.py</code>	Ejecuta comandos administrativos del proyecto.
<code>settings.py</code>	Configuración global.
<code>urls.py</code>	Rutas principales.
<code>asgi.py</code> / <code>wsgi.py</code>	Puntos de entrada para despliegue.

Paso 4: probar el proyecto inicial

Ejecutar migraciones iniciales.

```
python manage.py migrate
```

Levantar el servidor de desarrollo.

```
python manage.py runserver
```

Abrir en el navegador.

```
http://127.0.0.1:8000/
```

Antes de agregar aplicaciones, conviene confirmar que el proyecto base ejecuta correctamente y que la base de datos inicial fue creada.

Paso 5: crear carpeta de aplicaciones

Crear una carpeta para agrupar las aplicaciones del proyecto.

```
mkdir apps
```

Crear archivo de inicialización en Windows PowerShell.

```
New-Item apps/__init__.py -ItemType File
```

Crear archivo de inicialización en Linux o macOS.

```
touch apps/__init__.py
```

Estructura esperada:

```
proyecto_seguro/  
  apps/  
    __init__.py  
  config/  
  manage.py
```

Paso 6: crear la aplicación bienes

Crear únicamente la aplicación `bienes` .

```
python manage.py startapp bienes apps/bienes
```

Estructura generada:

```
apps/  
  bienes/  
    __init__.py  
    admin.py  
    apps.py  
    migrations/  
    models.py  
    tests.py  
    views.py
```

Paso 7: ajustar la configuración de la app

Revisar el archivo:

```
apps/bienes/apps.py
```

Debe quedar así:

```
from django.apps import AppConfig

class BienesConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "apps.bienes"
```

Como la app está dentro de la carpeta `apps/`, el nombre completo de importación es `apps.bienes`.

Paso 8: registrar la aplicación

Editar `config/settings.py` .

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
  
    # Aplicaciones del proyecto  
    "apps.bienes",  
]
```

Paso 9: configurar plantillas globales

En `config/settings.py`, buscar la sección `TEMPLATES`.

Cambiar:

```
"DIRS": [],
```

por:

```
"DIRS": [BASE_DIR / "templates"],
```

La sección debe quedar así:

```
TEMPLATES = [  
    {  
        "BACKEND": "django.template.backends.django.DjangoTemplates",  
        "DIRS": [BASE_DIR / "templates"],  
        "APP_DIRS": True,  
        "OPTIONS": {  
            "context_processors": [  
                "django.template.context_processors.request",  
                "django.contrib.auth.context_processors.auth",  
                "django.contrib.messages.context_processors.messages",  
            ],  
        },  
    ],  
]
```

Paso 10: configurar redirecciones de autenticación

Agregar al final de `config/settings.py` .

```
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"

LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "bienes:lista"
LOGOUT_REDIRECT_URL = "login"
```


Variable	Función
LOGIN_URL	Ruta a la que se envía al usuario si no ha iniciado sesión.
LOGIN_REDIRECT_URL	Ruta posterior a un inicio de sesión correcto.
LOGOUT_REDIRECT_URL	Ruta posterior al cierre de sesión.

Estas variables permiten controlar el flujo de entrada y salida del sistema sin escribir una vista manual de login.

Paso 11: crear rutas de la aplicación

Crear el archivo:

```
apps/bienes/urls.py
```

Contenido:

```
from django.urls import path
from . import views

app_name = "bienes"

urlpatterns = [
    path("", views.lista_bienes, name="lista"),
    path("nuevo/", views.crear_bien, name="crear"),
]
```

`app\_name = "bienes"` define el namespace lógico para usar rutas como `bienes:lista` y `bienes:crear`.

Paso 12: conectar rutas principales

Editar `config/urls.py`.

```
from django.contrib import admin
from django.urls import path, include
from django.contrib.auth import views as auth_views

urlpatterns = [
    path("admin/", admin.site.urls),

    path(
        "accounts/login/",
        auth_views.LoginView.as_view(
            template_name="registration/login.html"
        ),
        name="login",
    ),

    path(
        "accounts/logout/",
        auth_views.LogoutView.as_view(),
        name="logout",
    ),

    path("bienes/", include("apps.bienes.urls")),
]
```

Relación de rutas

Navegador

Solicita una URL del sistema.

config/urls.py

Recibe la ruta principal.

apps/bienes/urls.py

Resuelve la ruta del módulo.

views.py

Ejecuta la vista correspondiente.

El proyecto principal delega hacia la aplicación `bienes`, aunque todavía no exista registro de información.

Paso 13: crear vistas protegidas

Editar `apps/bienes/views.py` .

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render
```

```
@login_required
def lista_bienes(request):
    return render(request, "bienes/lista.html")
```

```
@login_required
def crear_bien(request):
    return render(request, "bienes/formulario.html")
```

El decorador `@login_required` exige que el usuario tenga sesión activa antes de acceder a la vista.

Lectura de las vistas

Elemento	Función
<code>login_required</code>	Restringe el acceso a usuarios autenticados.
<code>lista_bienes</code>	Representa la pantalla principal del módulo de bienes.
<code>crear_bien</code>	Representa una pantalla de formulario sin captura de datos.
<code>render()</code>	Devuelve una plantilla HTML al navegador.

En esta versión, las vistas solo validan acceso. No procesan formularios de datos ni guardan información.

Paso 14: crear directorios de plantillas

Crear la carpeta global de plantillas.

```
mkdir templates
```

Crear la carpeta para login.

```
mkdir templates\registration
```

Crear la carpeta de plantillas de la app.

```
mkdir apps\bienes\templates  
mkdir apps\bienes\templates\bienes
```

Estructura esperada:

```
templates/  
  base.html  
  registration/  
    login.html  
  
apps/  
  bienes/  
    templates/  
      bienes/  
        lista.html  
        formulario.html
```


Paso 15: crear plantilla base

Crear el archivo:

```
templates/base.html
```

Contenido:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}Proyecto seguro{% endblock %}</title>
</head>
<body>
  <header>
    <h1>Proyecto seguro en Django</h1>

    {% if user.is_authenticated %}
      <p>Usuario: {{ user.username }}</p>

      <form method="post" action="{% url 'logout' %}">
        {% csrf_token %}
        <button type="submit">Cerrar sesión</button>
      </form>
    {% endif %}
  </header>

  <main>
    {% block content %}{% endblock %}
  </main>
</body>
</html>
```

¿Qué valida la plantilla base?

La plantilla base no autentica por sí misma, pero sí adapta la interfaz al estado del usuario.

```
{% if user.is_authenticated %}  
  <p>Usuario: {{ user.username }}</p>  
{% endif %}
```

Caso	Resultado
Usuario autenticado	Muestra nombre de usuario y botón de cerrar sesión.
Usuario anónimo	No muestra opciones de sesión activa.

Ocultar contenido en la plantilla no protege la vista. La protección real se aplica con `@login_required`.

Paso 16: crear pantalla de login

Crear el archivo:

```
templates/registration/login.html
```

Contenido:

```
{% extends "base.html" %}

{% block title %}Iniciar sesión{% endblock %}

{% block content %}
<h2>Iniciar sesión</h2>

{% if form.errors %}
    <p>Usuario o contraseña inválidos.</p>
{% endif %}

<form method="post">
    {% csrf_token %}

    <p>
        <label for="{{ form.username.id_for_label }}">Usuario</label><br>
        {{ form.username }}
    </p>

    <p>
        <label for="{{ form.password.id_for_label }}">Contraseña</label><br>
        {{ form.password }}
    </p>

    <button type="submit">Entrar</button>
</form>
{% endblock %}
```

LoginView de Django

La pantalla visual es `login.html` , pero la lógica de autenticación la proporciona Django mediante `LoginView` .

```
from django.contrib.auth import views as auth_views
```

```
path(
    "accounts/login/",
    auth_views.LoginView.as_view(
        template_name="registration/login.html"
    ),
    name="login",
)
```

La autenticación debe apoyarse en el sistema de Django, no en validaciones manuales de usuario y contraseña dentro de vistas propias.

Paso 17: crear pantalla principal de bienes

Crear el archivo:

```
apps/bienes/templates/bienes/lista.html
```

Contenido:

```
{% extends "base.html" %}

{% block title %}Bienes{% endblock %}

{% block content %}
<h2>Módulo de bienes</h2>

<p>Esta pantalla representa la entrada protegida al módulo de bienes.</p>

<p>
  <a href="{% url 'bienes:crear' %}">Ir al formulario de bienes</a>
</p>
{% endblock %}
```

Paso 18: crear formulario vacío

Crear el archivo:

```
apps/bienes/templates/bienes/formulario.html
```


Contenido:

```
{% extends "base.html" %}

{% block title %}Formulario de bienes{% endblock %}

{% block content %}
<h2>Formulario de bienes</h2>

<p>En esta primera versión, el formulario todavía no captura datos del bien.</p>

<p>El objetivo es verificar que solo un usuario autenticado pueda acceder a esta pantalla.</p>

<p>
  <a href="{% url 'bienes:lista' %}">Volver al módulo de bienes</a>
</p>
{% endblock %}
```

El formulario está vacío porque en esta versión se valida acceso, no datos del dominio.

Paso 19: ejecutar revisión del proyecto

Ejecutar la revisión de configuración.

```
python manage.py check
```

Si no hay errores, ejecutar migraciones.

```
python manage.py migrate
```

Crear superusuario.

```
python manage.py createsuperuser
```

Levantar servidor.

```
python manage.py runserver
```

Paso 20: probar el administrador

Abrir:

```
http://127.0.0.1:8000/admin/
```

Entrar con el superusuario.

Desde el administrador se pueden crear usuarios adicionales para pruebas.

Usuario	Uso recomendado
Superusuario	Administración del sistema.
Usuario normal	Pruebas de acceso a pantallas protegidas.

El superusuario debe reservarse para administración. Las pruebas funcionales deben realizarse con usuarios de menor privilegio.

Paso 21: probar acceso sin sesión

Cerrar sesión o abrir una ventana privada del navegador.

Intentar entrar a:

```
http://127.0.0.1:8000/bienes/
```

Resultado esperado:

```
/accounts/login/?next=/bienes/
```

La redirección al login confirma que `@login\_required` está protegiendo la pantalla.

Paso 22: probar acceso con sesión

Iniciar sesión en:

```
http://127.0.0.1:8000/accounts/login/
```

Después de autenticarse correctamente, Django redirige a:

```
http://127.0.0.1:8000/bienes/
```

Desde ahí se puede acceder a:

```
http://127.0.0.1:8000/bienes/nuevo/
```

Resultado esperado:

- se muestra el usuario autenticado;
- aparece el botón de cerrar sesión;
- se puede navegar entre las pantallas de bienes;
- no se guarda información del dominio.

Flujo de autenticación construido

Usuario no autenticado

Intenta entrar a una pantalla protegida.

@login_required

Detecta que no existe sesión activa.

LoginView

Solicita usuario y contraseña.

Sesión

Django registra al usuario autenticado.

Vista protegida

El usuario accede al módulo de bienes.

Esta primera versión permite validar el acceso a pantallas antes de incorporar modelos, formularios y persistencia.

Estructura final de la versión mínima

```
proyecto_seguro/  
  manage.py  
  requirements.txt  
  
  config/  
    settings.py  
    urls.py  
  
  apps/  
    __init__.py  
    bienes/  
      __init__.py  
      apps.py  
      urls.py  
      views.py  
      templates/  
        bienes/  
          lista.html  
          formulario.html  
  
  templates/  
    base.html  
    registration/  
      login.html
```


Controles incorporados en esta versión

Control	Aplicación en el proyecto
Autenticación	Login con sistema de Django.
Sesión	Usuario autenticado disponible mediante <code>request.user</code> .
CSRF	Token en formularios <code>POST</code> , como login y logout.
Acceso protegido	Decorador <code>@login_required</code> .
Redirección controlada	<code>LOGIN_URL</code> , <code>LOGIN_REDIRECT_URL</code> , <code>LOGOUT_REDIRECT_URL</code> .
Administración	Creación de usuarios mediante <code>/admin/</code> .

Errores comunes que deben evitarse

Error	Consecuencia
Usar <code>app_name = "apps.bienes"</code>	Las rutas nombradas no funcionarán como se espera.
Omitir <code>include</code> en <code>config/urls.py</code>	No se podrán delegar rutas a la app.
No importar <code>auth_views</code>	Fallará la ruta de login o logout.
No crear <code>templates/registration/login.html</code>	Django no encontrará la plantilla de login.
Omitir <code>@login_required</code>	La pantalla quedará accesible sin sesión.
Confundir plantilla con protección real	Ocultar botones no bloquea el acceso a una vista.

Cierre de la versión mínima

La primera versión establece la base de autenticación y navegación protegida. Todavía no valida datos del dominio; solo valida si el usuario tiene una sesión activa.

En la siguiente versión se incorporará:

- modelo Bien ;
- formulario conectado al modelo;
- listado de registros;
- creación de bienes;
- migraciones propias de la aplicación;
- persistencia en SQLite.

Referencias de consulta

Recurso	Uso recomendado
Django admin y <code>manage.py</code>	Comandos de administración del proyecto.
Autenticación en Django	Login, usuarios, sesiones y permisos.
Configuración de Django	Parámetros como <code>LOGIN_URL</code> , <code>LOGIN_REDIRECT_URL</code> y plantillas.
URL dispatcher	Organización de rutas del proyecto y de la aplicación.
Templates	Herencia de plantillas y presentación HTML.

Gracias
¿Preguntas?

